SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

1. Testing as an Engineering Activity

Introduction: Software testing as an engineering activity is a systematic process used to identify defects and improve software quality.

1. Systematic Test Design:

- Prepare test cases based on requirements.
- Test valid and invalid inputs.
- Compare actual results with expected results.

2. Testing Tools:

- Selenium – for automated testing.
- Jira – for defect tracking.
- JUnit – for unit testing.

3. Testing Standards: Follow a defined testing process and standards such as ISO/IEC/IEEE 29119. Document test cases and defect reports properly.

4. Testing Techniques:

- Equivalence Partitioning
- Boundary Value Analysis
- Regression Testing
- Automation Testing

5. Practical Example: In a banking application, test a money transfer of ₹2,000 from an account containing ₹5,000. The expected balance is ₹3,000. Also test invalid amounts and duplicate transactions to identify defects.

Conclusion: Systematic testing, suitable tools, and standard procedures help detect defects early, reduce failures, and improve software quality.

2. Structured Testing Process in SDLC

Introduction: A structured testing process helps identify defects at every stage of the Software Development Life Cycle (SDLC) and improves software quality.

Stages of Testing:

1. Requirement Analysis: Understand the ATM withdrawal requirements.
2. Test Planning: Prepare the test strategy, resources, and schedule.
3. Test Case Design: Create test cases for valid and invalid withdrawals.
4. Test Execution: Execute test cases and compare actual results with expected results.
5. Defect Reporting: Record errors and report them to developers.
6. Retesting and Regression Testing: Verify fixes and ensure existing functions work correctly.

Practical Example: For an ATM withdrawal of ₹2,000 from an account with ₹5,000, the expected balance is ₹3,000. Test invalid PINs, insufficient balances, and withdrawals exceeding the daily limit.

Conclusion: Following a structured testing process reduces defects, improves consistency, and ensures reliable ATM operations.

3. Errors, Defects, and Failures

Introduction: Errors, defects, and failures are important concepts in software testing used to identify and resolve software problems.

1. Error: A human mistake made during coding or development.

2. Defect: A flaw in the software caused by an error.

3. Failure: When the software does not produce the expected result during execution.

Relationship: Error → Defect → Failure

Practical Example: In a web application, a developer mistakenly writes incorrect login validation logic (error). This creates a defect in the code. When a valid user cannot log in, it results in a failure.

Similarly, if a transaction displays an incorrect amount, the tester identifies and reports the defect.

Conclusion: Understanding errors, defects, and failures helps testers report problems clearly, identify root causes, and improve software quality.

4. Defect Clustering and Testing Principles

Introduction: Defect clustering means that most defects are found in a small number of software modules. Testing principles help prioritize testing and improve efficiency.

1. Testing Principles:

- Defect clustering: Focus on modules with more defects.
- Early testing: Identify defects early in development.
- Exhaustive testing is impossible: Prioritize important test cases.
- Risk-based testing: Test critical functions first.

2. Test Case Prioritization: Test high-risk modules, frequently used functions, and modules with a history of defects first.

Practical Example: In a banking application, if most defects occur in fund transfer, testers should prioritize transaction validation, balance checking, and duplicate transaction testing.

Conclusion: Applying defect clustering and testing principles helps detect critical defects early, save time, and improve software quality.

5. Defect Classification and Repository

Introduction: A defect repository is a centralized place to store defect details. Defect classification helps categorize defects and prevent them from recurring.

1. Defect Classification: Defects can be classified as:

- Functional defects
- Performance defects
- Security defects
- Interface defects
- Logical defects

2. Defect Repository: It stores defect descriptions, severity, priority, status, root cause, and resolution details.

3. Practical Example: If a banking application repeatedly deducts money twice during a transaction, the defect is recorded in the repository. Testers analyze the previous defect and create additional test cases to prevent recurrence.

4. Prevention:

- Maintain updated defect records.

- Analyze recurring defect patterns.
- Perform regression testing.
- Share defect information with developers and testers.

Conclusion: Defect classification and repositories help identify recurring problems, improve test design, prevent repeated defects, and enhance software quality.

6. Testing a Mobile Banking Application

Introduction: Systematic software testing helps identify the causes of failures and improves application reliability.

1. Identify Root Causes: Check coding errors, network issues, memory problems, and incorrect transaction handling.

2. Testing Approach:

- Perform functional testing of fund transfers.
- Test different network conditions.
- Check valid and invalid transaction amounts.
- Perform stress testing under heavy usage.
- Conduct regression testing after fixing defects.

3. Practical Example: If a mobile banking application crashes during a fund transfer, testers check whether the amount is deducted correctly and whether the transaction is completed only once. They identify the defect and report it to developers.

4. Prevention: Use automated testing, proper defect tracking, and regular regression testing to avoid future failures.

Conclusion: Systematic testing helps identify root causes, prevent transaction errors, reduce crashes, and improve the reliability of mobile banking applications.

7. Testing vs Debugging

Introduction: Testing and debugging are two important activities used to improve software quality.

1. Testing: Identifies defects by executing the software and comparing actual results with expected results.

2. Debugging: Finds the root cause of a defect and corrects the code.

Differences:

- Testing is performed to detect defects.
- Debugging is performed to locate and fix defects.
- Testing is usually done by testers, while debugging is commonly done by developers.

Practical Example: If a login page rejects a valid password, the tester identifies the failure and reports it. The developer debugs the code, finds the incorrect validation logic, and fixes it.

Conclusion: Testing and debugging work together to identify defects, correct errors, and improve software quality.

8. Cross-Browser Testing

Introduction: Cross-browser testing checks whether a web application works correctly across different browsers.

1. Testing Strategy:

- Test the application in Chrome, Firefox, and Edge.
- Check layout, buttons, forms, and login functionality.
- Verify compatibility across different screen sizes.
- Report and fix browser-specific defects.

2. Tools:

- Selenium – browser automation.
- BrowserStack – testing across browsers and devices.

3. Practical Example: An e-commerce website works correctly in Chrome but the payment button does not work in Firefox. Cross-browser testing helps identify and fix this defect.

Conclusion: Cross-browser testing improves compatibility, detects browser-specific defects, and ensures a consistent user experience.

9. Classification in a Payroll System

Introduction: Defect classification helps identify and categorize software defects to improve testing and prevent repeated errors.

1. Types of Defects:

- Functional defects
- Logical defects
- Performance defects
- Interface defects
- Data-related defects

2. Practical Example: In a payroll system, an employee's salary is calculated incorrectly due to a logical error in the tax calculation formula. The tester identifies and reports the defect.

3. Prevention:

- Validate salary calculation formulas.
- Test different salary ranges and tax values.
- Perform boundary value analysis.
- Conduct regression testing after fixing defects.

Conclusion: Defect classification helps identify the cause of errors, improve test coverage, and ensure accurate salary calculations.

10. Testing as an Engineering Activity

Introduction: Testing as an engineering activity is a systematic process that uses planned test cases and techniques to detect defects and improve software quality.

1. Structured Test Case Design:

- Understand the requirements.
- Prepare test cases with expected results.
- Test valid, invalid, and boundary values.
- Record and report defects.

2. Testing Techniques:

- Equivalence Partitioning
- Boundary Value Analysis
- Regression Testing
- Automation Testing

3. Practical Example: In a banking application, test withdrawals using valid amounts, insufficient balances, and amounts exceeding the withdrawal limit. Compare actual results with expected results to identify defects.

Conclusion: Structured test case design replaces random testing, improves defect detection, increases test coverage, and ensures reliable software.

11. Data Inconsistency in a Healthcare Application

Introduction: Software testing principles help identify defects and maintain data consistency across application modules.

1. Causes of Defects:

- Incorrect coding
- Database update errors

- Improper data validation
- Communication errors between modules

2. Testing Approach:

- Perform integration testing between modules.
- Validate database records.
- Check data accuracy and consistency.
- Report and track defects.

3. Practical Example: In a healthcare application, a patient's updated address appears in one module but not in another. Testers perform integration testing and identify the synchronization defect.

Conclusion: Systematic testing and proper defect tracking maintain data integrity, improve reliability, and ensure accurate healthcare information.

12. Defect Repository for Repeated Defects

Introduction: A defect repository stores information about software defects and helps teams analyze and prevent recurring problems.

1. Use of Defect Repository:

- Store defect descriptions and status.
- Record severity, priority, and root cause.
- Track defect fixes.
- Identify recurring defect patterns.

2. Practical Example: If a login defect occurs repeatedly across software releases, testers review previous defect records and create additional test cases to check login validation.

3. Prevention:

- Update the repository regularly.
- Analyze defect patterns.
- Perform regression testing.
- Share lessons learned with developers and testers.

Conclusion: A defect repository improves test case design, prevents recurring defects, and increases software quality.

13. Performance Testing of a Web-Based Learning System

Introduction: Software testing principles help identify performance issues and improve system reliability during heavy usage.

1. Testing Gaps:

- Lack of load testing
- Insufficient test coverage
- Untested peak-usage scenarios

2. Testing Approach:

- Perform load testing and stress testing.
- Prepare test cases for multiple users.
- Monitor response time and server performance.
- Identify and fix bottlenecks.

3. Practical Example: During an online examination, hundreds of students access the learning system simultaneously, causing it to crash. Load testing helps identify the performance problem before deployment.

Conclusion: Proper test planning, adequate coverage, and performance testing help prevent crashes and improve system reliability.

14. Role of Tester in SDLC

Introduction: A tester plays an important role in the Software Development Life Cycle (SDLC) by identifying defects and ensuring software quality.

1. Role of Tester:

- Review requirements.
- Prepare test plans and test cases.
- Execute tests and report defects.
- Verify fixes and perform regression testing.

2. Importance of Early Involvement: Early testing helps identify requirement errors, reduces correction costs, and prevents defects from reaching later stages.

3. Practical Example: In a banking application, a tester reviews fund transfer requirements during the design stage and identifies missing validation for insufficient balance.

4. Improvements:

- Involve testers from the requirement stage.
- Encourage communication between developers and testers.
- Conduct regular review meetings.

Conclusion: Early tester involvement improves collaboration, detects defects sooner, reduces development costs, and increases software quality.

15. Incomplete Test Coverage

Introduction: Test coverage measures how much of the software has been tested. Incomplete coverage may allow defects to remain undetected.

1. Causes:

- Missing test cases
- Untested inputs and conditions
- Ignoring boundary values
- Insufficient testing time

2. Improving Test Coverage:

- Prepare test cases for all requirements.
- Test valid, invalid, and boundary values.
- Use equivalence partitioning.
- Perform regression testing.

3. Practical Example: In a banking application, a withdrawal function works for normal amounts but fails when the account balance is zero. Testing boundary conditions helps identify this defect.

Conclusion: Proper test coverage improves defect detection, validates software functionality, and increases reliability.

16. Defect Reporting in a Logistics Application

Introduction: Defect reporting is the process of documenting software problems clearly so developers can identify and fix them.

1. Importance of Defect Reporting:

- Helps identify defects accurately.
- Improves communication between testers and developers.
- Supports defect tracking and resolution.

2. Elements of a Defect Report:

- Defect ID and title
- Steps to reproduce
- Expected and actual results

- Severity and priority
- Screenshots and status

3. Practical Example: In a logistics application, the delivery status does not update after a package is delivered. The tester records the steps, expected status, actual status, and screenshot in a defect report.

Conclusion: Clear and structured defect reports reduce resolution time, improve communication, and help maintain software quality.

17. Process-Based Testing Approach

Introduction: Process-based testing uses a standard and systematic procedure to maintain consistency across software modules.

1. Testing Approach:

- Prepare a standard test plan.
- Design test cases based on requirements.
- Follow the same testing procedures across modules.
- Record and report defects systematically.
- Perform regression testing after fixes.

2. Practical Example: In a banking application, different teams test login and fund transfer using the same documentation and defect-reporting process. This ensures consistent testing across modules.

3. Benefits:

- Improves testing consistency.
- Reduces repeated defects.
- Improves communication.
- Increases software quality.

Conclusion: A standardized testing process improves consistency, detects defects effectively, and ensures reliable software.

18. Boundary Value Analysis

Introduction: Boundary Value Analysis (BVA) is a testing technique that checks values at the boundaries of an input range because defects often occur at these limits.

1. Why Edge Cases Are Missed:

- Incomplete test cases

- Insufficient test coverage
- Testing only normal inputs

2. Testing Approach: Test minimum, maximum, just-below, and just-above boundary values.

3. Practical Example: If an application accepts ages from 18 to 60, test ages 17, 18, 19, 59, 60, and 61 to identify boundary-related defects.

4. Improvements:

- Include edge cases in test plans.
- Apply boundary value analysis.
- Perform regression testing.

Conclusion: Boundary Value Analysis helps detect edge-case defects, improves test coverage, and increases software reliability.

19. Defect Repository and Documentation

Introduction: A defect repository stores details of software defects and helps track, analyze, and resolve problems effectively.

1. Importance of Documentation:

- Records defect details clearly.
- Tracks defect status and resolution.
- Helps identify recurring defects.
- Improves communication between testers and developers.

2. Defect Repository: It contains defect ID, description, severity, priority, root cause, status, and solution.

3. Practical Example: In a banking application, a failed transaction is recorded in the defect repository with its error details and screenshots. Developers use this information to fix the problem, and testers verify the correction.

4. Improvements:

- Maintain updated defect records.
- Use tools such as Jira.
- Follow standard defect-reporting procedures.
- Analyze previous defects to prevent recurrence.

Conclusion: Proper defect documentation improves tracking, simplifies analysis, prevents repeated defects, and enhances software quality.

20. Testing a Banking Transaction System

Introduction: Systematic software testing helps identify transaction defects and ensures the accuracy and reliability of banking operations.

1. Testing Approach:

- Analyze requirements and prepare a test plan.
- Design test cases for valid and invalid transactions.
- Execute tests and compare expected and actual results.
- Report defects and verify fixes.
- Perform regression testing.

2. Testing Techniques:

- Functional Testing
- Boundary Value Analysis
- Integration Testing
- Regression Testing
- Automation Testing

3. Practical Example: During a fund transfer, a banking application deducts money twice. Testers reproduce the issue, report the defect, and verify the fix using regression testing to ensure that each transaction is processed only once.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand